

FeatureLeakageLens

Interview Defense Guide — v0.2.0

Pre-training feature leakage auditor for tabular ML datasets

1. What Is FeatureLeakageLens?

FeatureLeakageLens audits tabular ML datasets for suspicious feature leakage patterns **before model training**. It accepts a pandas DataFrame and a LeakageAuditConfig, runs six structured checks, and returns a LeakageReport with per-finding PASS / WARN / FAIL / INSUFFICIENT_INPUT status.

It is an **auditor, not a proof engine**. Every finding includes an explicit truth boundary: the tool flags suspicious patterns; the data scientist and domain expert confirm whether a feature was actually available at prediction time.

The six checks:

Check	What it detects
Post-outcome name heuristic	Column names containing post-outcome / target-proxy terms
Target correlation scan	Numeric features with $ \text{corr} \geq \text{threshold}$ vs target
Categorical proxy scan	Categorical features with large target-rate gap across values
Temporal availability	Feature timestamp after outcome timestamp (future leakage)
ID / proxy scan	High-cardinality identifier columns that memorize labels
Split distribution scan	Material train/test shift suggesting a leaky split

2. Algorithmic Implementations

2.1 Post-outcome Name Heuristic

Scans all column names (lowercased) against a keyword list: *paid, received, outcome, result, decision, approved, rejected, funded, default, label, target, flag, proxy, score, prediction* (and substrings thereof). Returns WARN for any match not equal to the declared target column. Rationale: post-outcome columns are often named descriptively and can be caught cheaply before any computation.

2.2 Target Correlation Scan

Computes Pearson correlation between each numeric feature and the binary target. Any feature with $|\text{r}| \geq \text{high_corr_threshold}$ (default 0.95) is flagged WARN. The check skips the target column itself and any column used as feature_time_col. High correlation does not prove leakage — it flags features for human review.

2.3 Categorical Proxy Scan

For each low-cardinality column ($n_{\text{unique}} / n_{\text{rows}} \leq 0.05$), computes the per-value target rate. If $\text{max_rate} - \text{min_rate} \geq \text{cat_proxy_threshold}$ (default 0.5), the column is flagged WARN. This catches categorical columns that perfectly partition the target — a common pattern when a post-processing

segment is accidentally included.

2.4 Temporal Availability

Given `outcome_time_col` and a `feature_time_cols` dict mapping feature column -> timestamp column, counts rows where `feature_ts > outcome_ts`. Any such row is a definitive future-leakage instance: the feature was generated after the label event. Returns FAIL (not WARN) because this is a structural violation, not just a statistical suspicion. Evidence dict includes `future_rows` count.

2.5 ID / Proxy Scan

Flags columns where $n_unique / n_rows \geq id_threshold$ (default 0.95) as high-cardinality identifiers. High-cardinality IDs can memorize training labels (the model uses the ID as a lookup key into training outcomes). Returns WARN so the data scientist can confirm whether the column is a meaningful feature or a row identifier that should be dropped.

2.6 Split Distribution Scan

Requires `split_col` with at least two distinct values (train/test). For numeric columns: computes normalized absolute mean difference ($|mean_train - mean_test| / pooled_std$). For categorical columns: computes Total Variation Distance ($TVD = 0.5 * \sum |p_train - p_test|$). Any column exceeding `distribution_shift_threshold` (default 0.1) is flagged WARN. Returns INSUFFICIENT_INPUT when no `split_col` is configured.

3. Status Rollup

The overall report status is the worst finding across all checks: **FAIL > WARN > INSUFFICIENT_INPUT > PASS**. A single FAIL (e.g., temporal availability) causes the whole report to be FAIL. INSUFFICIENT_INPUT means the check could not run due to missing config — it is not a clean bill of health.

Status	Meaning	Action
PASS	No suspicious patterns detected	Proceed, document the audit
WARN	Suspicious pattern — review required	Domain expert must confirm feature validity
FAIL	Structural violation (e.g., future timestamp)	Do not train until feature is removed or fixed
INSUFFICIENT_INPUT	Check skipped — missing config	Provide config fields to enable the check

4. Design Decisions

4.1 No model training required

All checks run against the raw DataFrame before any model is fit. This is intentional: leakage should be caught at data preparation time, not discovered via suspiciously high offline metrics.

4.2 FAIL vs WARN distinction

Temporal availability returns FAIL because future timestamps are a structural violation with no ambiguity — if a feature timestamp is after the outcome timestamp, that feature could not have been available at prediction time. All other checks return WARN because they are statistical patterns that require domain confirmation.

4.3 Thresholds are configurable

Every threshold (`high_corr_threshold`, `cat_proxy_threshold`, `id_threshold`, `distribution_shift_threshold`) is exposed in `LeakageAuditConfig` with sensible defaults. This avoids magic numbers and lets teams tune the auditor to their domain's risk tolerance.

4.4 Evidence dict on findings

Each `LeakageFinding` includes an evidence dict with quantitative details (e.g., `future_rows`, `correlation`, `target_rate_gap`). This makes reports actionable — the data scientist can immediately see how severe a finding is without re-running any code.

4.5 Three output formats

JSON for machine parsing and CI gates. Markdown for documentation and pull request comments. HTML for human review with highlighted findings. All three are written by `report.save(path)`.

4.6 Zero hard dependencies beyond pandas + numpy

The only runtime dependencies are pandas and numpy, which are already present in any tabular ML workflow. This makes `FeatureLeakageLens` installable as a pre-training step without introducing dependency conflicts.

5. Hard Interview Questions

Q: Pearson correlation only detects linear relationships. What leakage patterns does it miss?

A: Correct — Pearson r will miss a monotonic but nonlinear proxy (e.g., a log-transformed target), a categorical feature that encodes target perfectly (caught by the categorical proxy check), and a feature with conditional leakage (leaks only within a segment). The tool uses correlation as a cheap first-pass signal, not a complete leakage proof. The categorical proxy check and the name heuristic cover complementary patterns. The truth boundary in every report makes this explicit.

Q: The categorical proxy scan uses max_rate - min_rate. Is that robust to class imbalance?

A: It is sensitive to imbalance in the expected direction: a 90/10 imbalanced target will have a majority class rate near 0.9 in most bins, so the gap is naturally smaller unless the feature truly partitions the target. The threshold (default 0.5) was chosen conservatively for this reason. An alternative is to normalize by overall target rate (lift), which would be more sensitive; that is a documented roadmap item.

Q: Why not use mutual information instead of Pearson correlation for the target scan?

A: Mutual information is more general but requires density estimation or discretization, which introduces hyperparameters (n_bins, estimator type) and is sensitive to sample size. Pearson correlation is O(n), parameter-free, and sufficient to flag near-perfect linear proxies which are the most common leakage pattern. Mutual information is a documented roadmap extension for nonlinear leakage detection.

Q: The ID/proxy scan flags high-cardinality columns. Wouldn't that also flag legitimate features like zip code or product SKU?

A: Yes, intentionally. A zip code or SKU is legitimately high-cardinality but may still memorize training labels in a small dataset. The check returns WARN, not FAIL, so the data scientist can confirm whether the column is a valid feature (appropriate for the model) or a row identifier that should be excluded. The truth boundary is explicit.

Q: The split distribution scan uses pooled std for numeric shift. What if one split is empty?

A: The check guards against empty splits: if either train or test subset has zero rows, the finding is INSUFFICIENT_INPUT for that column. A zero-row split would cause a division-by-zero in pooled std computation, so the guard prevents a runtime error and returns a clear status.

Q: Can this tool be used as a CI gate to block model training?

A: Yes. The report.status field is PASS / WARN / FAIL / INSUFFICIENT_INPUT and the JSON output is machine-parseable. A CI script can call audit_dataframe(), check report.status == 'FAIL', and exit non-zero to block the training pipeline. WARN is a softer gate — teams can choose to require human sign-off on WARN findings before proceeding.

Q: Why is temporal leakage FAIL while correlation leakage is only WARN?

A: Temporal leakage is a structural violation with no ambiguity: if a feature's timestamp is after the outcome timestamp on even one row, that feature was definitionally unavailable at prediction time. There is no domain-specific exception. Correlation leakage is a statistical pattern that requires domain context: a feature may be highly correlated with the target legitimately (e.g., a strong predictor). Only a domain expert can confirm whether it is a valid feature or a proxy.

Q: What does INSUFFICIENT_INPUT mean? Is it better or worse than WARN?

A: INSUFFICIENT_INPUT means the check could not run because required config was absent (e.g., no split_col for the distribution scan, no feature_time_cols for temporal check). It is not a clean bill of health — it means the audit is incomplete. In the rollup, INSUFFICIENT_INPUT ranks below WARN but above PASS, so a report with only INSUFFICIENT_INPUT findings returns status

INSUFFICIENT_INPUT, not PASS.

Q: How does this handle the case where the same column appears both as a feature and as a timestamp?

A: feature_time_cols maps feature column name -> timestamp column name. The timestamp column is used purely for temporal comparison and is excluded from correlation/categorical scans to avoid false positives (a raw datetime column would show up as high-cardinality or uncorrelated noise in those checks).

6. Truth Boundary

FeatureLeakageLens does not:

- Prove that a feature caused model inflation — it flags patterns for human review
- Replace feature store governance, data contracts, or production monitoring
- Detect all leakage patterns (nonlinear proxies, multi-feature interactions, indirect paths)
- Make automatic go/no-go decisions — a data scientist must review findings

FeatureLeakageLens does:

- Systematically audit six known leakage patterns before a single model is trained
- Return structured, machine-parseable findings with quantitative evidence
- Make the audit step documentable, repeatable, and reviewable
- Surface definitive temporal violations (FAIL) vs statistical suspicions (WARN)

7. Files Index

File	Purpose
src/featureleakagelens/core.py	Main audit logic — 6 checks, LeakageAuditConfig, LeakageFinding, LeakageReport
tests/test_featureleakagelens.py	13 unit tests across 4 test classes
scripts/generate_demo_reports.py	Demo script — generates JSON/MD/HTML outputs
data/demo_leakage_dataset.csv	Demo dataset with synthetic leakage patterns
README.md	Badges, arch diagram, About, check table, quickstart
CHANGELOG.md	v0.1.0 and v0.2.0 release notes
METHODOLOGY.md	Full check derivations and design rationale
pyproject.toml	Package config — version 0.2.0
.github/workflows/ci.yml	Python 3.10/3.11/3.12 CI matrix
.github/workflows/publish.yml	OIDC PyPI trusted publisher on release

Resume-safe claim: Built **FeatureLeakageLens**, a pre-training feature leakage auditor for tabular ML datasets that checks for post-outcome name heuristics, target correlation, categorical target-rate proxies, future timestamp leakage, ID/proxy columns, and train/test distribution shift, producing structured JSON/Markdown/HTML audit reports with per-finding PASS/WARN/FAIL/INSUFFICIENT_INPUT status and explicit truth boundary.